

重庆三峡科技大学数学与统计学院

课程综合考查报告

课程名称： 专业实习

题 目： 基于 YOLO 的车牌检测与猫狗识别系统

学 生： 何慧娴

学 号： 202306174204

专业班级： 2023 级大数据管理与应用 2 班

课程时间： 2026. 5. 6-2025. 5. 16

目 录

第 1 章 引言	3
1.1 项目设计需求	3
1.1.1 车牌检测需求	3
1.1.2 猫狗识别需求	3
1.2 项目功能要求	3
1.2.1 界面友好性要求	4
1.2.2 系统响应要求	4
第 2 章 项目设计方案	4
2.1 开发平台	4
2.1.1 开发工具	4
2.1.2 运行环境	4
2.2 开发语言与框架	4
2.2.1 后端开发语言	4
2.2.2 Web 框架	5
2.2.3 前端技术	5
2.3 硬件开发平台	5
2.4 设计实现方案	5
2.4.1 系统总体架构	5
2.4.2 技术方案选择	5
2.5 软件流程图	6
第 3 章 项目设计实现	7
3.1 主程序设计	7
3.1.1 图片处理函数	7
3.1.2 检测响应格式	7
3.2 模块设计	7
3.2.1 模块一：车牌检测模块	7
3.2.2 模块二：猫狗识别模块	8
3.3 程序运行效果	9
3.3.1 效果一：系统首页	9
3.3.2 效果二：车牌检测效果	10
3.3.3 效果三：猫狗识别效果	10
第 4 章 总结与体会	12

第 1 章 引言

1.1 项目设计需求

随着人工智能技术的快速发展，计算机视觉在日常生活中的应用越来越广泛。车牌识别技术在智能交通系统、停车场管理、电子警察等场景中发挥着重要作用；而猫狗识别技术则在宠物管理、动物行为分析、智能摄像头等领域具有广阔的应用前景。

本项目旨在设计并实现一个基于 YOLO 深度学习模型的车牌检测与猫狗识别系统。系统采用 B/S 架构，用户可通过浏览器上传图片，后端利用训练好的 YOLO 模型进行实时检测，并返回检测结果（包括目标位置边界框、置信度、类别信息等）。

1.1.1 车牌检测需求

车牌检测是智能交通系统的核心技术之一。本项目的车牌检测模块需要能够从复杂背景中准确识别出车牌位置，并用边界框标注出来。系统应支持实时检测，检测结果需包含车牌位置坐标和置信度。

1.1.2 猫狗识别需求

猫狗识别模块需要能够检测图像中是否包含猫或狗，并准确识别其种类（猫或狗），同时标注出目标在图像中的位置。该模块可用于宠物监控、动物分类等场景。

1.2 项目功能要求

- 1.双功能并行检测：**系统同时支持车牌检测和猫狗识别两个功能，左右布局，互不干扰。
- 2.图片上传功能：**用户可通过点击上传按钮选择本地图片，并在页面上实时预览。
- 3.后端检测功能：**系统后端集成 YOLO 模型，能够对上传的图片进行目标检测，返回标注后的结果图片和检测信息。
- 4.结果可视化：**检测结果包括边界框、类别标签和置信度，并在图片上直观展示。
- 5.信息反馈：**系统需向用户反馈检测结果，包括检测到的目标数量、每个目标的类别、置信度和位置信息。

1.2.1 界面友好性要求

系统界面应简洁美观，操作便捷。采用左右分栏布局，左侧为车牌检测模块，右侧为猫狗识别模块，每个模块均有独立的图片上传区域、检测按钮和结果显示区域。

1.2.2 系统响应要求

检测响应时间应控制在合理范围内，单张图片检测时间不超过 3 秒，保证用户体验流畅。

第 2 章 项目设计方案

2.1 开发平台

本项目采用 Windows 操作系统为开发平台。Windows 平台具有良好的兼容性和丰富的开发工具支持，便于 Django 框架的部署和调试。

2.1.1 开发工具

IDE: PyCharm

测试工具: Django 内置开发服务器

2.1.2 运行环境

操作系统: Windows

浏览器: Edge

2.2 开发语言与框架

2.2.1 后端开发语言

项目后端使用 Python 语言进行开发。Python 具有丰富的第三方库支持，尤其在深度学习领域具有明显优势。

2.2.2 Web 框架

采用 Django 框架作为 Web 后端框架。Django 是一个高级 Python Web 框架，提供了 ORM、路由、模板引擎等完整功能，能够快速构建稳定的 Web 应用。

后端主要依赖库：

django: Web 框架

ultralytics: YOLO 模型加载和推理

2.2.3 前端技术

前端使用原生 HTML、CSS 和 JavaScript 实现。CSS 实现现代化的界面样式，包括圆角、阴影、渐变、响应式布局等效果；JavaScript 负责图片上传预览、AJAX 请求发送、加载动画显示和结果渲染。

2.3 硬件开发平台

模型训练主要依赖 CPU 加速。

2.4 设计实施方案

2.4.1 系统总体架构

系统采用 B/S（Browser/Server）架构，分为前端展示层、后端业务层和模型推理层三个层次：

前端展示层：用户交互界面，负责图片上传、检测请求发送和结果展示。

后端业务层：Django 视图函数，处理 HTTP 请求，调用模型进行推理，返回 JSON 响应。

模型推理层：加载预训练的 YOLO 模型，对输入图片执行目标检测。

2.4.2 技术方案选择

车牌检测模型：使用 CCPD 数据集预训练的 YOLO 模型（ccpd_best.pt）。CCPD（Chinese City Parking Dataset）是一个大规模的中国城市停车场车牌数据集，包含大量不同场景下的车牌图片。

猫狗识别模型:使用 Kaggle 猫狗数据集训练的自定义 YOLO 模型(cat_dog_best.pt)。该数据集包含猫和狗的多角度、多姿态图片，模型经过 60 轮训练，能够准确识别猫和狗的位置及种类。

检测流程:

前端上传图片 → 转换为 Base64 格式 → 发送 POST 请求

后端接收 → 解码为 OpenCV 图像 → 保存临时文件

YOLO 模型推理 → 获取检测结果（边界框、类别、置信度）

绘制标注框 → 保存结果图片 → 返回 URL 和检测信息

2.5 软件流程图



第 3 章 项目设计实现

3.1 主程序设计

项目主程序主要包括 Django 项目配置、路由配置和视图函数实现三部分。

3.1.1 图片处理函数

为了处理前端传来的 Base64 格式图片，实现了 `get_image_array` 工具函数，将 Base64 字符串转换为 OpenCV 图像数组。

3.1.2 检测响应格式

后端统一返回 JSON 格式数据，包含以下字段：

`status`: 状态码（200 成功，500 失败）

`processed_img_url`: 处理后图片的 URL

`message`: 提示信息

`type`: 检测类型标识（"01-car_id" / "02-cat_dog"）

`detections`: 检测结果列表（包含类别、置信度、边界框）

3.2 模块设计

3.2.1 模块一：车牌检测模块

车牌检测模块负责识别图像中的车牌位置。使用 CCPD 数据集训练的 YOLO 模型，该数据集包含多种场景下的车牌图片（不同光照、角度、遮挡等）。

检测结果示例：

类别: `car_id`（车牌）

输出: 车牌边界框坐标 `[x1, y1, x2, y2]`



3.2.2 模块二：猫狗识别模块

猫狗识别模块负责检测图像中的猫和狗，并区分种类。使用 Kaggle 猫狗数据集训练的 YOLO 模型。

检测类别：

class_id	类别名称	说明
0	cat	猫
1	dog	狗

训练配置：

数据集来源：Kaggle - Dogs vs Cats

训练轮数：60 epochs

批次大小：4
图像尺寸：320×320

02 🐱🐶 猫狗识别（位置+种类）

原始图片

检测结果

🐾 识别猫狗位置和种类

📄 检测结果

类型: 02-cat dog

消息: 检测到 1 个动物

02-dog 🐶 识别为: 狗狗置信度: 88.3%

位置边框: [149.1, 71, 316.3, 206.7]

3.3 程序运行效果

3.3.1 效果一：系统首页

🚗🐱🐶 智能检测系统

01-车牌识别 | 02-猫狗识别（位置+种类） — 左右布局，同时操作

01 🚗 车牌识别检测

原始图片

📷

点击上传图片

检测结果

🔍

预测结果

🔍 开始识别车牌

02 🐱🐶 猫狗识别（位置+种类）

原始图片

📷

点击上传猫/狗图片

检测结果

🎯

识别结果（位置+种类）

🐾 识别猫狗位置和种类

3.3.2 效果二：车牌检测效果

上传包含车牌的图片，点击"开始识别车牌"按钮，系统返回标注了车牌位置的图片，并显示检测结果信息。

检测结果示例：

检测到 1 个车牌

置信度：78.0%

位置：[62, 504.7, 602.3, 631.3]。



3.3.3 效果三：猫狗识别效果

上传猫或狗的图片，点击"识别猫狗位置和种类"按钮，系统返回标注了目标位置的图片，并明确显示是猫还是狗。

检测结果示例：

识别为：猫咪 /狗

02 🐱🐶 猫狗识别（位置+种类）



🐱🐶 识别猫狗位置和种类

📄 检测结果

类型: 02-cat_dog
消息: 检测到 1 个动物

02-cat 🐱 识别为: 猫咪置信度: 97.3%
位置边框: [9.5, 15.6, 227.4, 239.6]

02 🐱🐶 猫狗识别（位置+种类）



🐱🐶 识别猫狗位置和种类

📄 检测结果

类型: 02-cat_dog
消息: 检测到 1 个动物

02-dog 🐶 识别为: 狗狗置信度: 94.2%
位置边框: [190.9, 41.8, 358.1, 331.9]

第 4 章 总结与体会

项目总结

本项目成功实现了一个基于 YOLO 算法的车牌检测与猫狗识别 Web 系统。系统采用 Django 框架作为后端，集成了训练好的 YOLO 模型，能够对用户上传的图片进行实时目标检测，并返回可视化结果。

主要完成工作：

模型训练：使用 Kaggle 猫狗数据集，训练了 YOLO 猫狗识别模型，训练 60 轮，图像尺寸 320×320 ，批次大小 4。

Web 系统开发：基于 Django 框架开发了完整的 Web 应用，包括前端界面（HTML/CSS/JS）和后端逻辑（视图函数、模型加载、图片处理）。

双功能并行：系统同时支持车牌检测和猫狗识别两个功能，采用左右分栏布局，界面简洁友好。

实时检测：单张图片检测时间控制在 2-3 秒内，满足实时性要求。

结果可视化：检测结果包括边界框标注和详细信息展示，用户体验良好。

心得体会

理论与实践结合：将课堂学习的深度学习理论（YOLO 算法、目标检测原理）应用到实际项目中，加深了对知识的理解。

全栈开发能力：掌握了从模型训练到 Web 部署的完整流程，提升了综合开发能力。

问题解决能力：在开发过程中遇到环境配置、模型加载、前后端通信等问题，通过查阅资料和调试逐一解决。

工具使用熟练度：熟练掌握了 Django 框架、Ultralytics YOLO 库、OpenCV 等工具的使用。

项目规划意识：学会了合理规划项目结构、分模块开发、逐步测试的工程方法。

未来改进方向

模型优化：尝试使用 YOLOv9 或 YOLOv10 等更新版本的模型，提高检测精度和速度。

数据集扩充：收集更多场景下的数据，增强模型的泛化能力。

实时视频检测：增加摄像头实时视频流检测功能。

多语言支持：增加英文界面，扩大使用范围

